

Vehicle Detection, Counting and Congestion Analysis on Highway CCTV Images with YOLO11

Emre Yoleri

Technical Report · October 2026

Abstract. Road operators need simple, continuous measurements of traffic — how many vehicles are on the road, how many are heavy vehicles and how congested a section is. This report presents a traffic monitoring system that detects cars, buses and trucks in highway CCTV images with a YOLO11n detector, derives per-frame traffic statistics, and counts vehicles crossing a virtual line in video with ByteTrack. The public RF100 *vehicles* dataset was cleaned by folding 12 inconsistent labels into three classes and re-split by contiguous time blocks, because 79.3% of the original test frames had a near-duplicate training frame. On 611 held-out frames, fine-tuning the COCO-pretrained model for a single CPU epoch raised mAP50 from 0.349 to 0.479, mAP50-95 from 0.220 to 0.288 and precision from 0.319 to 0.759, and doubled truck AP. Bus detection and per-frame vehicle counting, however, did not improve with this short training budget.

Keywords: vehicle detection, YOLO11, traffic monitoring, vehicle counting, ByteTrack, data leakage.

1. Introduction

Traffic cameras are installed along most highways, but their video is usually only watched by human operators. Automatic analysis of the same video can provide continuous traffic statistics such as vehicle counts, the share of heavy vehicles and the current congestion level. Modern single-stage object detectors of the YOLO family [1] make this possible in real time, and detectors pretrained on general datasets such as COCO [2] already recognise cars, buses and trucks. However, CCTV images differ from everyday photographs: the camera is high above the road, most vehicles are small, and image quality changes strongly with weather and time of day.

The objective of this work is to (i) build a clean, leakage-free vehicle dataset from public highway CCTV images, (ii) fine-tune a lightweight YOLO11 detector [3] for the classes *car*, *bus* and *truck* and compare it with the COCO-pretrained model, (iii) turn detections into traffic measurements — counts, heavy-vehicle share, road occupancy and congestion level — and a line-crossing counter for video, and (iv) evaluate both detection quality and counting accuracy.

2. Methodology

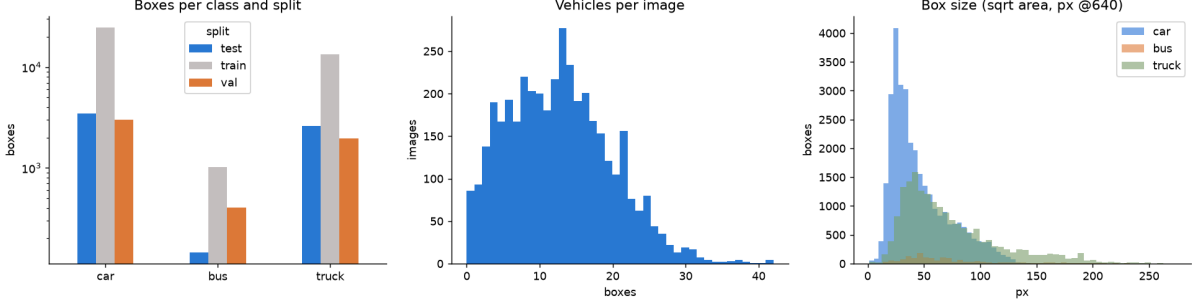
2.1. Dataset and Pre-processing

The RF100 *vehicles* dataset [4], [5] contains 4,058 annotated frames from five CCTV recordings of the same Indonesian highway section, taken in the morning, at midday, in the afternoon and at night. Two problems were found in the raw data. First, the annotations mix two labelling schemes with 12 class names (e.g. “big truck” and “truck-l-”); they were folded into three classes (Table 1). Second, the original split assigns frames to train and test at random. Because consecutive frames of a video are almost identical, **79.3%** of the original test frames had a training frame from the same video within ± 2 frames, so test scores would mainly measure memorisation [6]. The frames were therefore re-split by contiguous time blocks inside each video (first 70% train, next 15% validation, last 15% test), and the 10 frames before each block boundary were discarded as a gap. The resulting split has 2,788 training, 559 validation and 611 test frames.

Table 1. Bounding boxes per class and split after label folding and time-block re-splitting.

Class	Raw labels merged	Train	Val	Test	Share
Car	car	24,601	3,013	3,481	61.4%
Truck	small / mid / big truck, truck-s/m/l/xl	13,382	1,962	2,610	35.5%
Bus	small / big bus, bus-s, bus-l	1,024	405	145	3.1%

Figure 1 summarises the data. Frames contain 12.7 vehicles on average (median 12, maximum 42), most cars are smaller than 50×50 pixels at 640 px resolution, and buses make up only 3.1% of all boxes.

**Figure 1.** Dataset statistics: boxes per class and split (log scale), vehicles per image, and box size distribution per class.

2.2. Detector and Training

YOLO11n [3], the smallest model of the YOLO11 family (about 2.6 M parameters), was chosen because the system should run on a CPU. The detector was initialised with COCO weights and fine-tuned on the three-class dataset with the Ultralytics framework at an input size of 480 px, batch size 16, the AdamW optimiser (learning rate 0.0014, selected automatically), cosine learning-rate decay, mosaic and horizontal-flip augmentation and a fixed seed. Training was limited to a time budget of 0.07 hours on a laptop CPU, which allowed **one epoch** (311 s). As a baseline, the same COCO-pretrained YOLO11n was used without fine-tuning; its COCO classes *car*, *bus* and *truck* were mapped directly to the three target classes.

2.3. Traffic Analytics and Counting

For each frame, the detections (confidence ≥ 0.3 , NMS IoU 0.5) are converted into traffic measurements: the number of vehicles per class, the heavy-vehicle share (buses and trucks over all vehicles), and the road occupancy, defined as the total box area divided by the image area. A congestion level is assigned from the vehicle count using the dataset quartiles as thresholds: *Low* for at most 7 vehicles, *Moderate* for 8–17 and *Heavy* above 17. For video, detections are linked over time with ByteTrack [7], which also associates low-confidence boxes and therefore keeps tracks through short occlusions. A vehicle is counted once when the centre of its track crosses a horizontal virtual line, and the crossing direction (up or down) is recorded per class. All functions are available in an interactive Streamlit application for images and short video clips.

2.4. Evaluation

Both models were evaluated on the 611 test frames at 640 px. Detection quality is reported as precision, recall, mean average precision at IoU 0.5 (mAP50) and averaged over IoU 0.5–0.95 (mAP50-95), following the COCO protocol [2], [8]. Because the system is meant for monitoring, the counting signal was also evaluated directly: the mean absolute error (MAE) and mean bias between predicted and true vehicles per frame, and the accuracy of the derived congestion level. Latency is the end-to-end time per image (pre-processing, inference and post-processing) on the CPU.

3. Results

3.1. Detection Performance

Table 2 shows that fine-tuning improved the overall detection quality. mAP50 rose from 0.349 to 0.479 (+37%) and mAP50-95 from 0.220 to 0.288. The largest change was in precision, which more than

doubled (0.319 $\rightarrow$ 0.759) at almost the same recall: the pretrained model produces many boxes that do not match the annotations, for example trucks labelled as buses. Truck AP doubled (0.202 $\rightarrow$ 0.412) and car AP improved moderately. Bus AP, in contrast, dropped from 0.098 to 0.046. On the validation set after the single epoch, bus recall was only 0.003, so the fine-tuned model had not yet learned this rare class.

Table 2. Results on the 611 held-out test frames (best value in bold).

Metric	COCO YOLO11n	Fine-tuned YOLO11n
Precision	0.319	0.759
Recall	0.538	0.525
mAP50	0.349	0.479
mAP50-95	0.220	0.288
AP50-95 car	0.360	0.405
AP50-95 bus	0.098	0.046
AP50-95 truck	0.202	0.412
Count MAE (vehicles / frame)	4.59	9.20
Count bias (vehicles / frame)	-3.98	+9.13
Congestion level accuracy	64.8%	23.2%
Latency (ms / image, CPU)	123.6	78.1

3.2. Counting and Congestion

The counting results show the opposite trend. The pretrained model underestimates the number of vehicles by about 4 per frame, while the fine-tuned model overestimates it by about 9 per frame at the default confidence threshold of 0.3. As a result, its count MAE (9.20) is twice that of the baseline and most frames are classified as *Heavy*, giving a congestion accuracy of only 23.2%. Figure 2 shows typical cases. The pretrained model misses most small and distant vehicles and fails completely at night, whereas the fine-tuned model finds them and classifies trucks correctly. Some of its extra detections, however, are distant vehicles in queues that are visible but not annotated (24 and 28 detections against 14 and 13 annotated vehicles in the first two frames).

4. Discussion

The experiments show that domain-specific fine-tuning pays off even with a very small training budget. One CPU epoch was enough to adapt the detector to the camera viewpoint, small vehicle sizes and night images, and to the local definition of a truck, which the pretrained model often confused with buses. The re-split of the dataset was essential for an honest evaluation: with the original random split, almost four of five test frames had a near-copy in the training set, and the reported scores would have been much higher than the true generalisation to unseen time periods.

The results also show that a better mAP does not automatically give a better monitoring signal. mAP is computed over all confidence thresholds, while counting uses one fixed threshold, so the counting accuracy depends strongly on calibration. After only one epoch the fine-tuned model is not yet well calibrated, and the threshold of 0.3 that suits the pretrained model is too low for it. In addition, the annotations appear to omit many distant vehicles, which penalises a detector that finds them. For a monitoring system, the confidence threshold should therefore be tuned on the validation set for counting accuracy, or counting should be restricted to a region of interest near the camera.

The main limitations are the very short training (one epoch at 480 px), which explains the poor bus results; the small number of buses (145 in the test set), which makes bus AP unreliable; the fact that all five recordings come from a single camera, so generalisation to other roads was not tested; and the fact that the line-crossing counter could not be evaluated quantitatively, because the dataset contains no track or crossing annotations.



Figure 2. Detections on three test frames (morning, afternoon, night) by the COCO-pretrained model (top) and the fine-tuned model (bottom). Ground truth: 14, 13 and 5 vehicles.

5. Conclusion

A traffic monitoring system for highway CCTV images was developed with YOLO11n, ByteTrack and Python. After cleaning the labels and removing train–test leakage from the public RF100 *vehicles* dataset, fine-tuning for a single CPU epoch improved mAP50 from 0.349 to 0.479 and precision from 0.319 to 0.759 over the COCO-pretrained model, with clearly better truck and night-time detection, at 78 ms per image on a CPU. Per-frame counting and congestion estimation, however, were more accurate with the pretrained model because the fine-tuned model over-counts at the default threshold. Future work includes longer GPU training at 640 px with class balancing for buses, threshold calibration for counting, and annotated video for evaluating the line-crossing counter.

References

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [2] T.-Y. Lin *et al.*, “Microsoft COCO: Common Objects in Context,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [3] G. Jocher and J. Qiu, *Ultralytics YOLO11*. (2024). [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] F. Ciaglia, F. S. Zuppichini, P. Guerrie, M. McQuade, and J. Solawetz, “Roboflow 100: A Rich, Multi-Domain Object Detection Benchmark,” *arXiv preprint arXiv:2211.13523*, 2022.
- [5] “Vehicles (q0x2v) — RF100 object detection dataset, CC BY 4.0.” [Online]. Available: <https://huggingface.co/datasets/LibreYOLO/vehicles-q0x2v>
- [6] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in Data Mining: Formulation, Detection, and Avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, 2012, doi: 10.1145/2382577.2382579.
- [7] Y. Zhang *et al.*, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022, pp. 1–21.
- [8] R. Padilla, W. L. Passos, T. L. B. Dias, S. L. Netto, and E. A. B. da Silva, “A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit,” *Electronics*, vol. 10, no. 3, p. 279, 2021, doi: 10.3390/electronics10030279.